UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
DISCIPLINA: SISTEMAS DIGITAIS
Prof. Fernanda Lima Kastensmidt

**2026/2**

**Trabalho 1 – Sistemas Digitais - Individual**
**Aluna: Thais Marcelle Dihl da Silva - 00341545**

# Projeto do Processador Neander em VHDL

## 1) Descrição do trabalho

O projeto consiste na implementação do Neander com a instrução SUB. Na realização do RTL, cada componente foi feito separadamente, deixando o processador mais modular e fácil de testar para o caso de inconsistências.

Com relação à estrutura, foram feitos acumulador, program counter, registrador de instrução, registrador de endereço de memória e de dados, registrador de flags, multiplexador, decoder, unidade de controle e ULA. Cada componente foi testado individualmente por meio de testbenches e posteriormente integrado ao processador.

Para a instrução SUB, foi escolhido o opcode 0111, ela utiliza o mesmo formato de operando do ADD e realiza a operação usando complemento de dois, conforme solicitado, sendo implementada na ULA por meio da operação com tipos `signed`, como mostrado no trecho de código abaixo:

```
      case op is
           --soma pois pode produzir um resultado de 9 bits
           when "000" =>
               soma := unsigned('0' & a ) + unsigned('0' & b);
               resultado := std_logic_vector(soma(7 downto 0));
               --sub com complemento, pois usando a lib numeric e signed, já
realiza com o complemento de 2
           when "001" =>
               resultado:= std_logic_vector(signed(a)-signed(b));
```

Para a estrutura da memória, foi utilizado o Block Memory Generator do Vivado e o formato foi: memória Single Port RAM, Largura de 8 bits, 256 posições, leitura síncrona com latência de um ciclo, modo de escrita write first e habilitação da memória por meio do sinal ena.

Por fim, foram desenvolvidos dois programas em Assembly, carregados na BRAM por meio de arquivos COE: subtração de duas matrizes 3×3 e multiplicação por somas sucessivas.

## 2) VHDL completo do Neander

-- Código completo está no fim como anexo, esse é o Neander.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity neander is
    port (
        clk     : in std_logic;
        rst     : in std_logic;
```

```
        o_ac    : out std_logic_vector(7 downto 0); --val atual do acc
        o_pc    : out std_logic_vector(7 downto 0); --end atual do pc
        o_flags : out std_logic_vector(1 downto 0);
        o_hlt   : out std_logic  -- reconhecimento da instr hlt
    );
end entity neander;

architecture estrutural of neander is

    ---SINAIS INTERNOS
    signal pc_out, rem_in, rem_out         : std_logic_vector(7 downto 0);
    signal rdm_in, rdm_out, mem_out        : std_logic_vector(7 downto 0);
    signal ri_out, ac_out, ula_out         : std_logic_vector(7 downto 0);

    signal N_ula, Z_ula                    : std_logic;
    signal N_flag,Z_flag                   : std_logic;

    signal is_nop, is_sta, is_lda : std_logic;
    signal is_add, is_or, is_and  : std_logic;
    signal is_not, is_sub, is_jmp : std_logic;
    signal is_jn, is_jz, is_hlt  : std_logic;

    signal select_PC, PC_mais_um : std_logic;
    signal load_ri               : std_logic;
    signal op_ac                 : std_logic_vector(1 downto 0);
    signal op_ula                : std_logic_vector(2 downto 0);
    signal loadN, loadZ          : std_logic;
    signal mem_en, mem_we        : std_logic;
    signal load_rem, sel_rem_mux : std_logic;
    signal load_rdm, sel_rdm_mux : std_logic;
    signal wea_bram              : std_logic_vector(0 downto 0);
begin

-- Copia o valor de mem_we para a posição 0 do vetor wea_bram.
    wea_bram(0) <= mem_we;
-- cop valores p visualizar na analise
    o_ac    <= ac_out;
    o_pc    <= pc_out;
    o_flags <= N_flag & Z_flag;
    o_hlt   <= is_hlt;


    --armazena o end da prox inst, recebe novo end ou incrementa
    u_pc: entity work.PC
        port map (
            clk => clk,
            rst => rst,
            select_PC => select_PC,
            PC_mais_um => PC_mais_um,
            in_pc => rdm_out,
            out_pc => pc_out
        );
    -- mux do reg de end de mem
    -- 0 -> seleciona o PC
    -- 1 -> seleciona o RDM
    -- pode receber tanto o end da prox inst quanto  end lido da memoria
    u_mux_rem: entity work.mux2p1_8bits
        port map (
            a => pc_out,
```

```
            b => rdm_out,
            sel_mux => sel_rem_mux,
            mux_out => rem_in
        );
    --   armazena o end utilizado p acessar uma posicao da BRAM
    --   entrada recebe end selecionado pelo mux rem
    u_rem: entity work.reg_rem
        port map (
            clk => clk,
            rst => rst,
            data_in => rem_in,
            load_rem => load_rem,
            out_rem => rem_out
        );
    -- memoria BRAM
    u_memoria: entity work.memoria_neander
        port map (
            clka => clk,
            ena => mem_en, -- habilita acesso
            wea => wea_bram, --habilita escrita
            addra => rem_out,
            dina => rdm_out,
            douta => mem_out
        );
    -- 0-> seleciona a saida da BRAM
    -- 1 -> seleciona o valor do acumulador
     -- permite carregar dados lidos da mesm ou preparar um dado p ser escrito
nela
    u_mux_rdm: entity work.mux2p1_8bits
        port map (
            a => mem_out,
            b => ac_out,
            sel_mux => sel_rdm_mux,
            mux_out => rdm_in
        );
    -- armazena temporariamente os dados lidos da BRAM ou selecionados
    -- pelo multiplexador do RDM.
    -- Sua saida pode alimentar outros registradores, a ULA e o PC.
    u_rdm: entity work.reg_rdm
        port map (
            clk => clk,
            rst => rst,
            data_in => rdm_in,
            load_rdm => load_rdm,
            out_rdm => rdm_out
        );
    -- armazena inst buscada na mem, 4 bits mais sign contem o opcode
    u_ri: entity work.ri
        port map (
            clk => clk,
            rst => rst,
            data_in => rdm_out,
            load_ri => load_ri,
            out_ri => ri_out
        );
    -- recebe o opcode do RI, identifica a instrucao e ativa as saidas
    u_decoder: entity work.decoder
        port map (
            opcode => ri_out(7 downto 4),
```

```
            is_nop => is_nop,
            is_sta => is_sta,
            is_lda => is_lda,
            is_add => is_add,
            is_or => is_or,
            is_and => is_and,
            is_not => is_not,
            is_sub => is_sub,
            is_jmp => is_jmp,
            is_jn => is_jn,
            is_jz => is_jz,
            is_hlt => is_hlt
        );
    -- a -> dado do acumulador, b -> dado do RDM
    u_ula: entity work.ULA
        port map (
            a => ac_out,
            b => rdm_out,
            op => op_ula,
            rst => ula_out,
            N => N_ula,
            Z => Z_ula
        );
     -- acumulador armazena as op realizadas pela ula ou dados carregados da
memoria
    u_ac: entity work.ac
        port map (
            clk => clk,
            rst => rst,
            data_mem => rdm_out,
            data_ula => ula_out,
            op => op_ac,
            out_ac => ac_out
        );
    -- armazena flags da ula
    u_flags: entity work.flags
        port map (
            clk => clk,
            rst => rst,
            N_in => N_ula,
            Z_in => Z_ula,
            loadN => loadN,
            loadZ => loadZ,
            Z_out => Z_flag,
            N_out => N_flag
        );
     --recebe clk, rst, flags e inst do decoder e gera os sinais de habilitacao
dos regs,
    -- selecao dos mux, operacao da ula e controle de leitura e escrita
    u_controle: entity work.controle
        port map (
            clk => clk,
            rst => rst,
            N_flag => N_flag,
            Z_flag => Z_flag,
            is_nop => is_nop,
            is_sta => is_sta,
            is_lda => is_lda,
            is_add => is_add,
```

```
            is_or => is_or,
            is_and => is_and,
            is_not => is_not,
            is_sub => is_sub,
            is_jmp => is_jmp,
            is_jn => is_jn,
            is_jz => is_jz,
            is_hlt => is_hlt,
            select_PC => select_PC,
            PC_mais_um => PC_mais_um,
            load_ri => load_ri,
            op_ac => op_ac,
            op_ula => op_ula,
            loadN => loadN,
            loadZ => loadZ,
            mem_en => mem_en,
            mem_we => mem_we,
            load_rem => load_rem,
            sel_rem_mux => sel_rem_mux,
            load_rdm => load_rdm,
            sel_rdm_mux => sel_rdm_mux
        );

end architecture estrutural;
```

3) Testbench VHDL completo

```
-- Código completo está no fim como anexo, esse é o TB_neander
library ieee;
use ieee.std_logic_1164.all;

entity tb_neander is
end tb_neander;

architecture tb of tb_neander is

    component neander
        port (
            clk     : in std_logic;
            rst     : in std_logic;
            o_ac    : out std_logic_vector(7 downto 0);
            o_pc    : out std_logic_vector(7 downto 0);
            o_flags : out std_logic_vector(1 downto 0);
            o_hlt   : out std_logic
        );
    end component;

    signal clk     : std_logic := '0';
    signal rst     : std_logic := '1';

    -- Clock de 50 MHz
    constant TbPeriod : time := 20 ns;

    signal TbClock    : std_logic := '0';
    signal TbSimEnded : std_logic := '0';

    -- Saidas de depuracao do Neander
    signal o_ac_tb    : std_logic_vector(7 downto 0);
    signal o_pc_tb    : std_logic_vector(7 downto 0);
    signal o_flags_tb : std_logic_vector(1 downto 0);
```

```
    signal o_hlt_tb   : std_logic;
    signal ciclos_execucao : natural := 0;
    signal contagem_ativa : std_logic := '0';
    signal contagem_finalizada : std_logic := '0';
begin

    dut : neander
    port map (
        clk     => clk,
        rst     => rst,
        o_ac    => o_ac_tb,
        o_pc    => o_pc_tb,
        o_flags => o_flags_tb,
        o_hlt   => o_hlt_tb
    );

    -- Geracao do clock
    TbClock <= not TbClock after TbPeriod/2
        when TbSimEnded /= '1' else '0';

    clk <= TbClock;
-- Contador de ciclos de clock ate o HLT
    conta_ciclos : process(clk)
        begin
            if rising_edge(clk) then

                if rst = '1' then
                    ciclos_execucao <= 0;
                    contagem_ativa <= '0';
                    contagem_finalizada <= '0';

                elsif contagem_finalizada = '0' then

                    -- Inicia a contagem apos a liberacao do reset
                    if contagem_ativa = '0' then
                        contagem_ativa <= '1';
                        ciclos_execucao <= 1;

                    -- Encerra a contagem quando HLT e reconhecido
                    elsif o_hlt_tb = '1' then

                        report "TOTAL DE CICLOS DE CLOCK = " &
                               integer'image(ciclos_execucao + 1)
                               severity note;

                        contagem_ativa <= '0';
                        contagem_finalizada <= '1';

                    else
                        ciclos_execucao <= ciclos_execucao + 1;
                    end if;

                end if;
            end if;
        end process;
    estimulos : process
    begin

        -- Reset
```

```
        rst <= '1';
        wait for 5 * TbPeriod;

        -- Libera o processador
        rst <= '0';

        -- Aguarda a execucao do programa
        wait for 500 * TbPeriod;

        report "Fim da simulacao do Neander."
            severity note;

        TbSimEnded <= '1';
        wait;

    end process;

end architecture tb;
```

4) Explicação e descrição das aplicações em Assembly

**4.1 Subtração de duas matrizes A e B 3x3 com dados de 8 bits, onde os dados das matrizes estão armazenados em memória.**

Para gerar os arquivos .coe, foi feito um script python que traduzisse do ASM para .coe.

Como foi implementado a operação de SUB, no código assembly foram realizadas várias subtrações, obedecendo a forma que fazendo subtração de matrizes . No caso, Carrega no lado do item da matriz X, subtrai pelo equivalente da matriz Y e guarda esse valor em resultado.

```
Matriz X:    08   05   02
             07   04   01
             06   03   04

Matriz Y:    08   01  00
             05  05   04
             04  02   02

Matriz resultante: 00 04 02
                   02 FF FD
                   02 01 02
Programa em Assembly:
        NOP
        LDA A
        SUB J
        STA RESULTA
        LDA B
        SUB K
        STA RESULTB
        LDA C
        SUB L
        STA RESULTC
        LDA D
        SUB M
        STA RESULTD
        LDA E
```

```
        SUB N
        STA RESULTE
        LDA F
        SUB O
        STA RESULTF
        LDA G
        SUB P
        STA RESULTG
        LDA H
        SUB Q
        STA RESULTH
        LDA I
        SUB R
        STA RESULTI
        HLT
.ORG 0x40
;MATRIZ X
A:   .BYTE 08
B:   .BYTE 07
C:   .BYTE 06

D:   .BYTE 05
E:   .BYTE 04
F:   .BYTE 03

G:   .BYTE 02
H:   .BYTE 01
I:   .BYTE 04
;MATRIZ Y
J:   .BYTE 08
K:   .BYTE 05
L:   .BYTE 04

M:   .BYTE 01
N:   .BYTE 05
O:   .BYTE 02

P:   .BYTE 00
Q:   .BYTE 04
R:   .BYTE 02
;MATRIZ RESULTADO
RESULTA: .BYTE 00
RESULTB: .BYTE 00
RESULTC: .BYTE 00

RESULTD: .BYTE 00
RESULTE: .BYTE 00
RESULTF: .BYTE 00

RESULTG: .BYTE 00
RESULTH: .BYTE 00
RESULTI: .BYTE 00
        .END
```

**4.2 Multiplicação de dois valores A e B por soma sucessiva**

Na multiplicação, foi feito um Loop que roda B vezes, até ele ser zero, quando B for zero, carrega o resultado da multiplicação em RESULT.

```
; A = 4, B = 3. Resultado esperado = 12 (0C hexadecimal).
        NOP
```

```
        LDA ZERO
        STA RESULT
        LDA B
        JZ FIM

LOOP:
        LDA RESULT
        ADD A
        STA RESULT
        LDA B
        SUB UM
        STA B
        JZ FIM
        JMP LOOP

FIM:
        LDA RESULT
        HLT
.ORG 1C
A:      .BYTE 04
B:      .BYTE 03
RESULT: .BYTE 00
ZERO:   .BYTE 00
UM:     .BYTE 01
.END
```

5) Simulações sem e com atraso com detalhes e flechas mostrando inicio meio e final do programa e resultados

Figura 1 - Início da execução da subtração de matrizes, imagem mostrando a inicialização e busca da primeira instrução

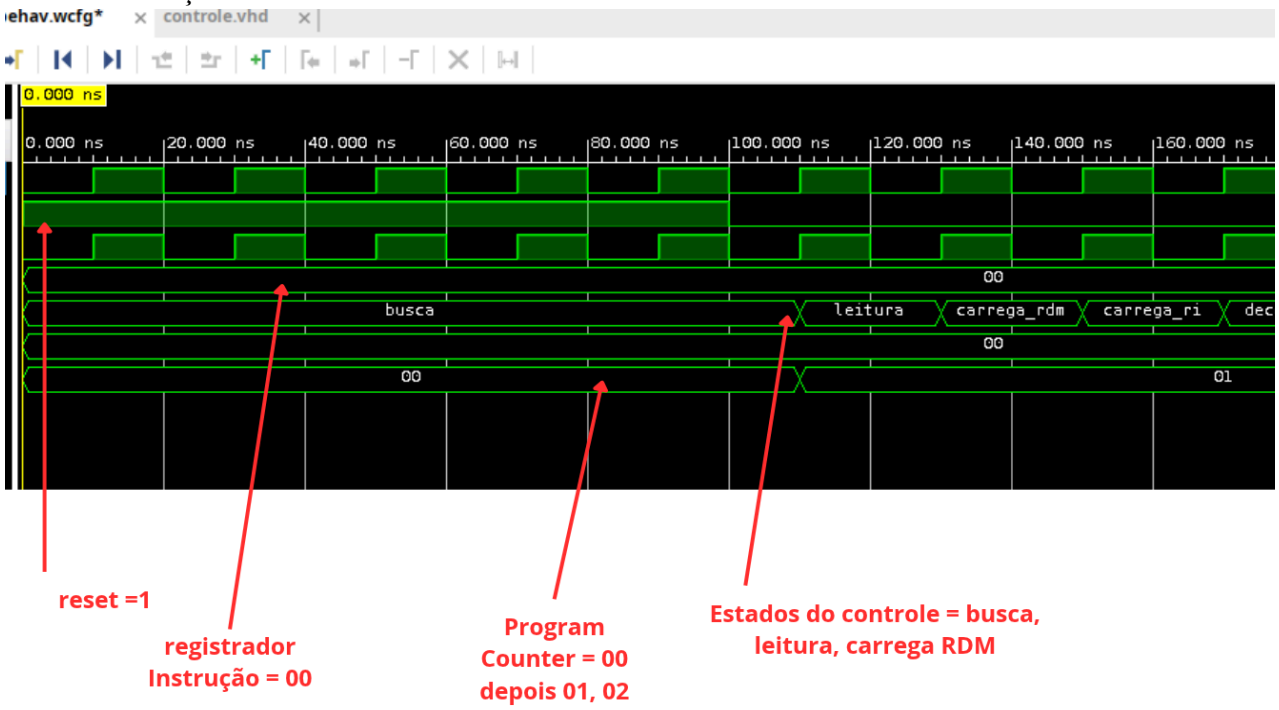


Figura 2 - Meio da execução, mostrando a operação E - N e o resultado FF em complemento de dois.

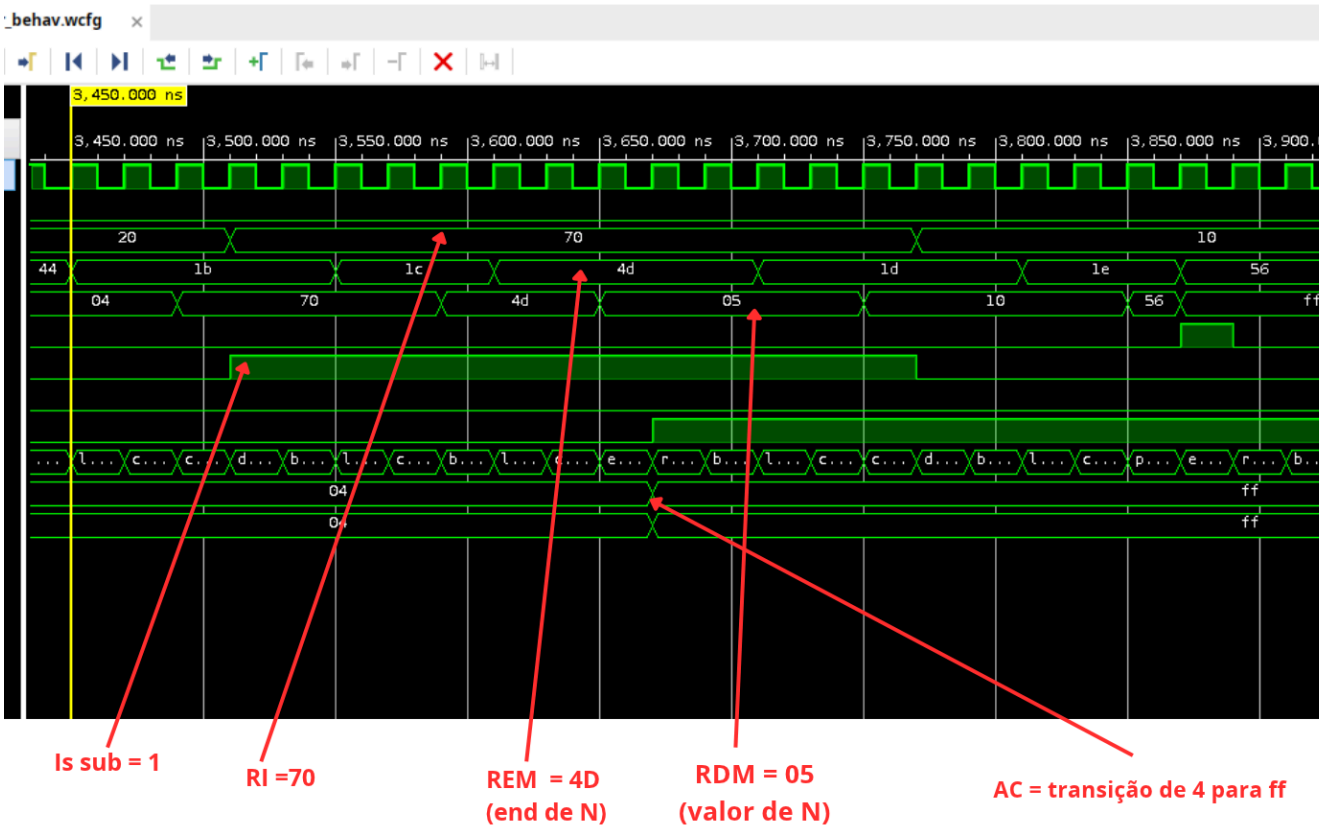



Figura 3 -Final da execução com o resultado final de 02 no acumulador e o reconhecimento da HLT

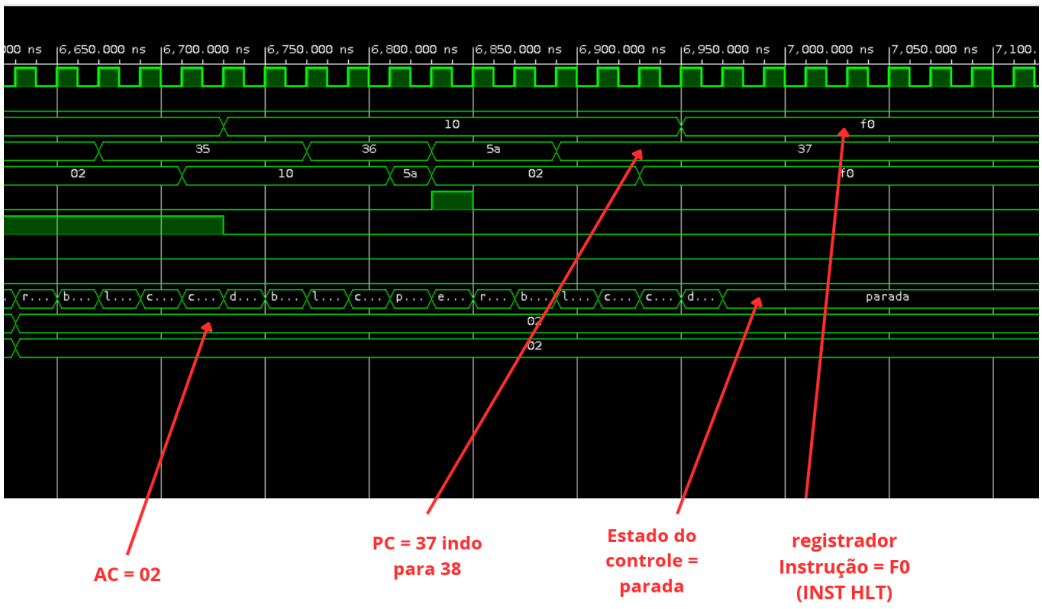


Figura 4 - Início da multiplicação por soma sucessiva, demonstrando a primeira soma do valor A no acumulador.

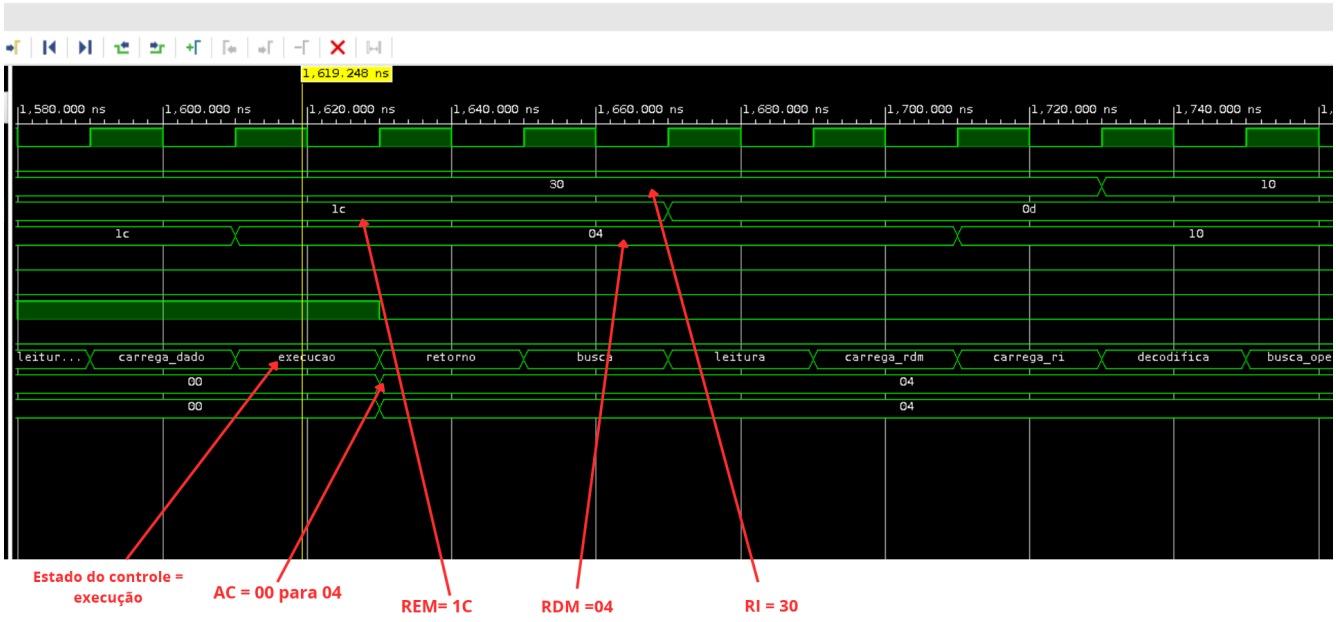



Figura 5 - Etapa final do laço da multiplicação, mostrando o acumulador de 08 para OC (12 em decimal) após a última soma.

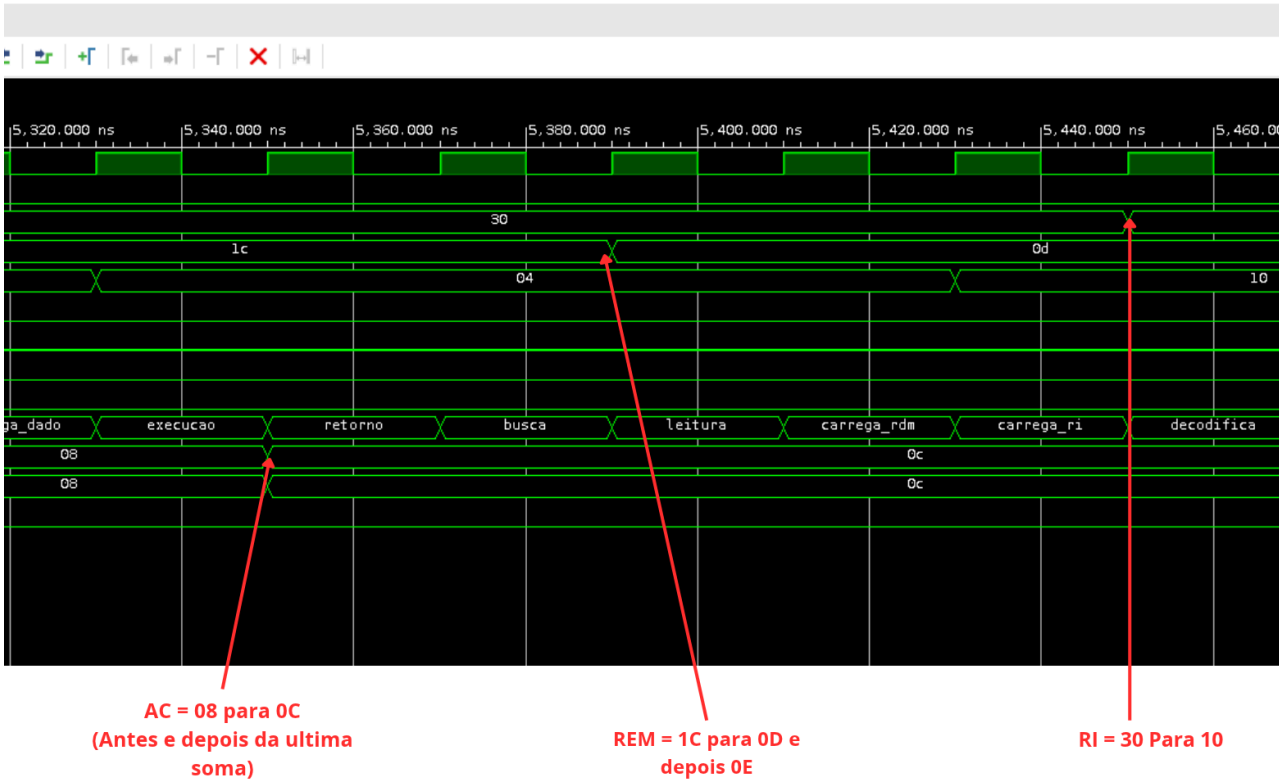


Figura 6 - Finalização da multiplicação por somas sucessivas, o resultado OC no acumulador e HLT 1.

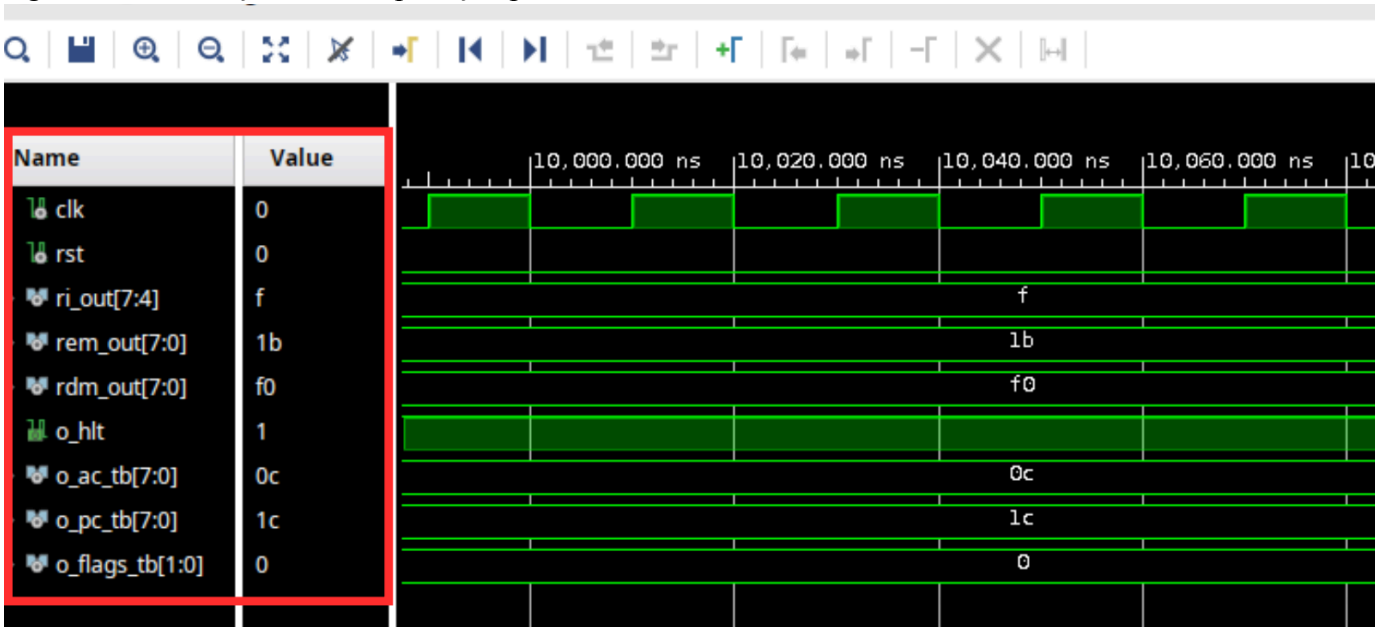



Figura 7 - Início da simulação temporal pós-implementação do programa de subtração de matrizes.

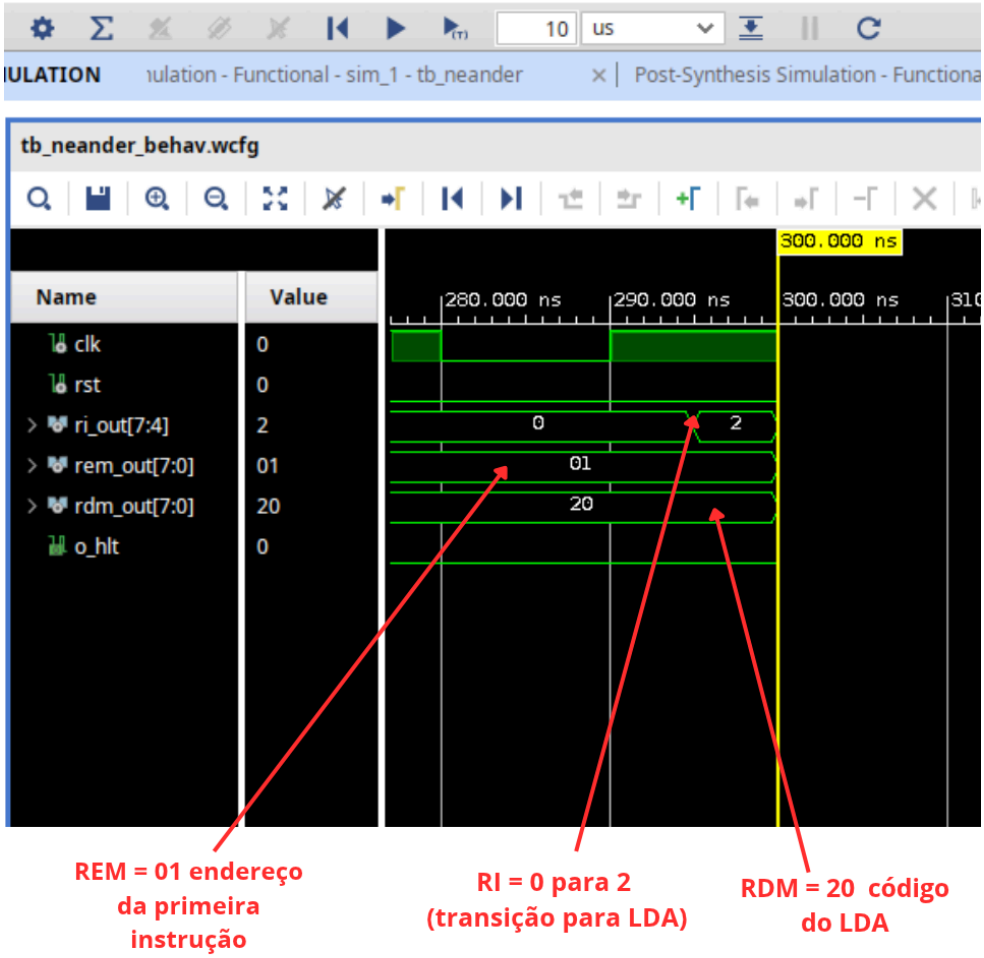


Figura 8 - Etapa intermediária da simulação temporal da subtração, destacando a leitura do operando e a execução da instrução SUB.

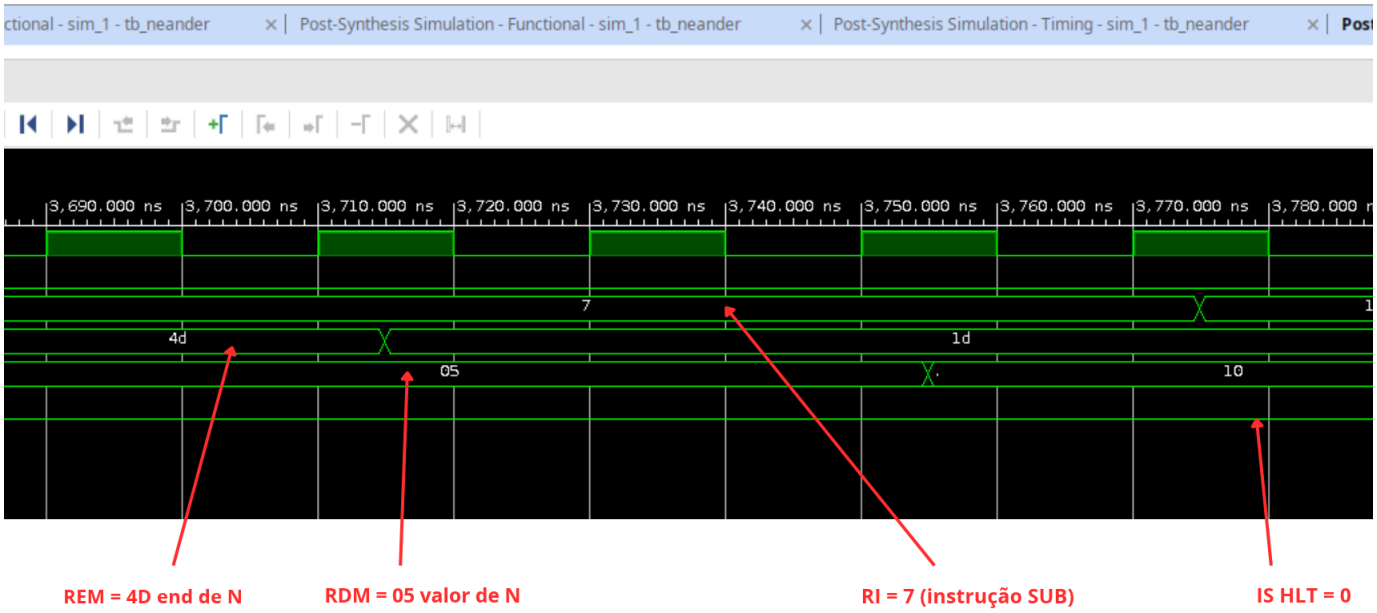



Figura 9 - Finalização da simulação temporal pós-implementação da subtração de matrizes, com resultado 02 no acumulador e reconhecimento de HLT.

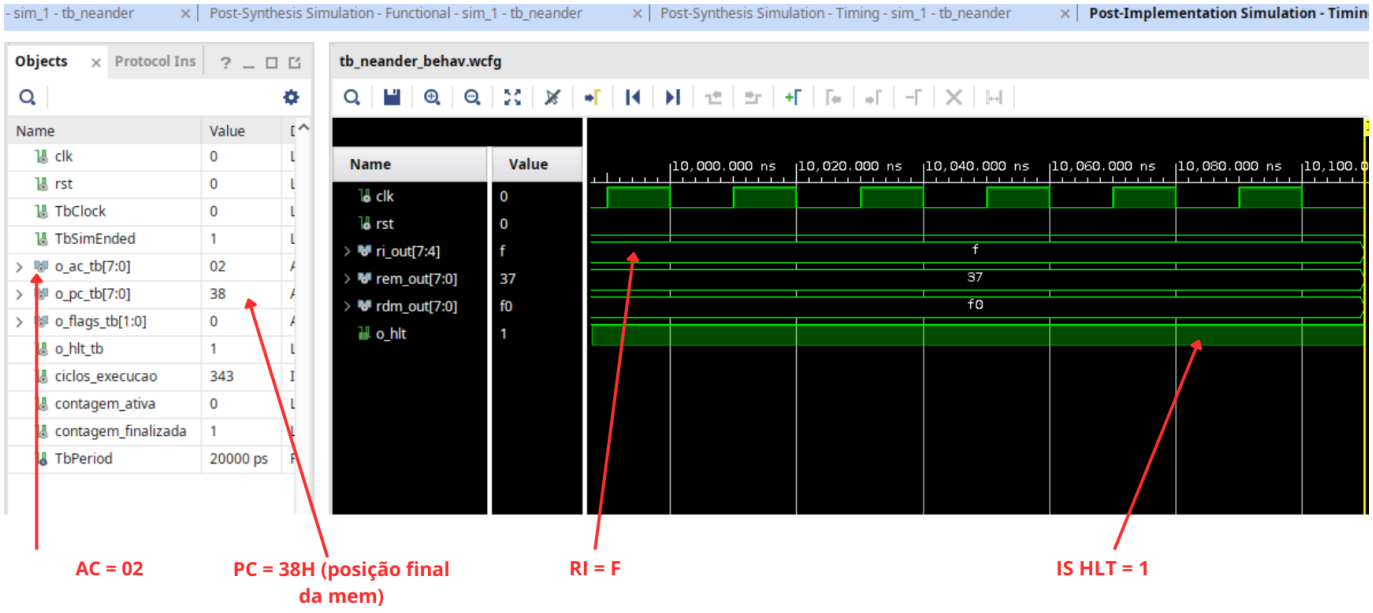


Figura 10 — Início da simulação temporal pós-implementação do programa de multiplicação por somas sucessivas.

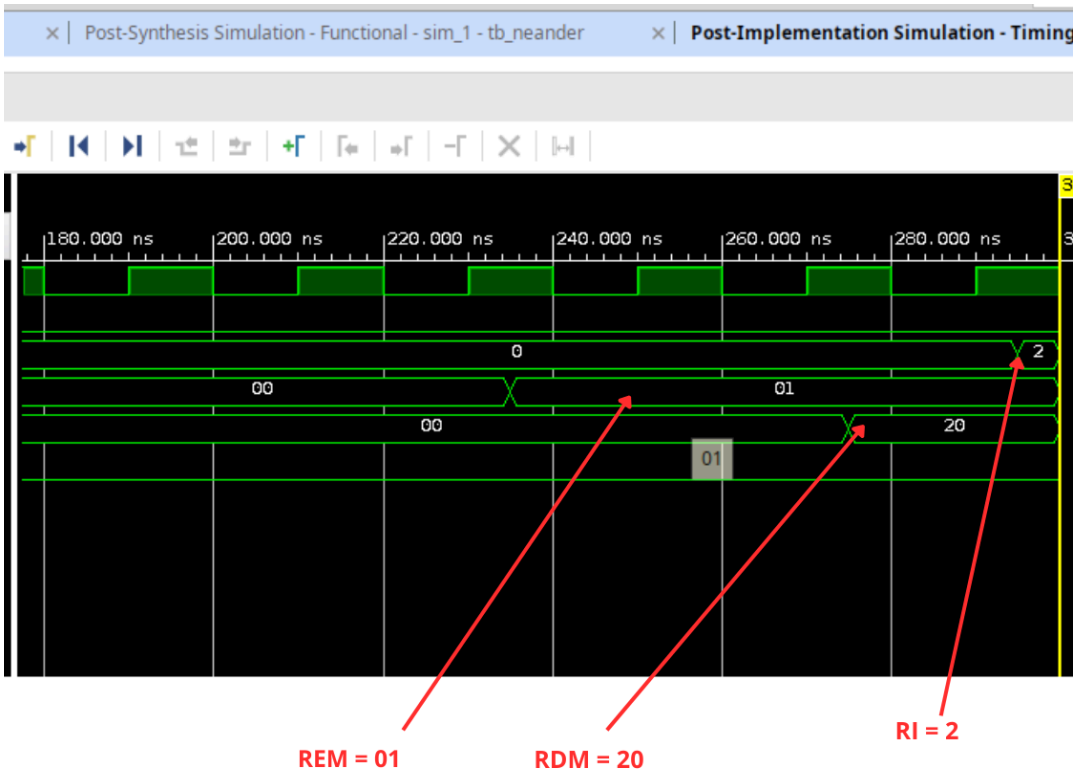



Figura 11 — Etapa intermediária da simulação temporal da multiplicação, apresentando o resultado parcial 08 no acumulador após a segunda soma.

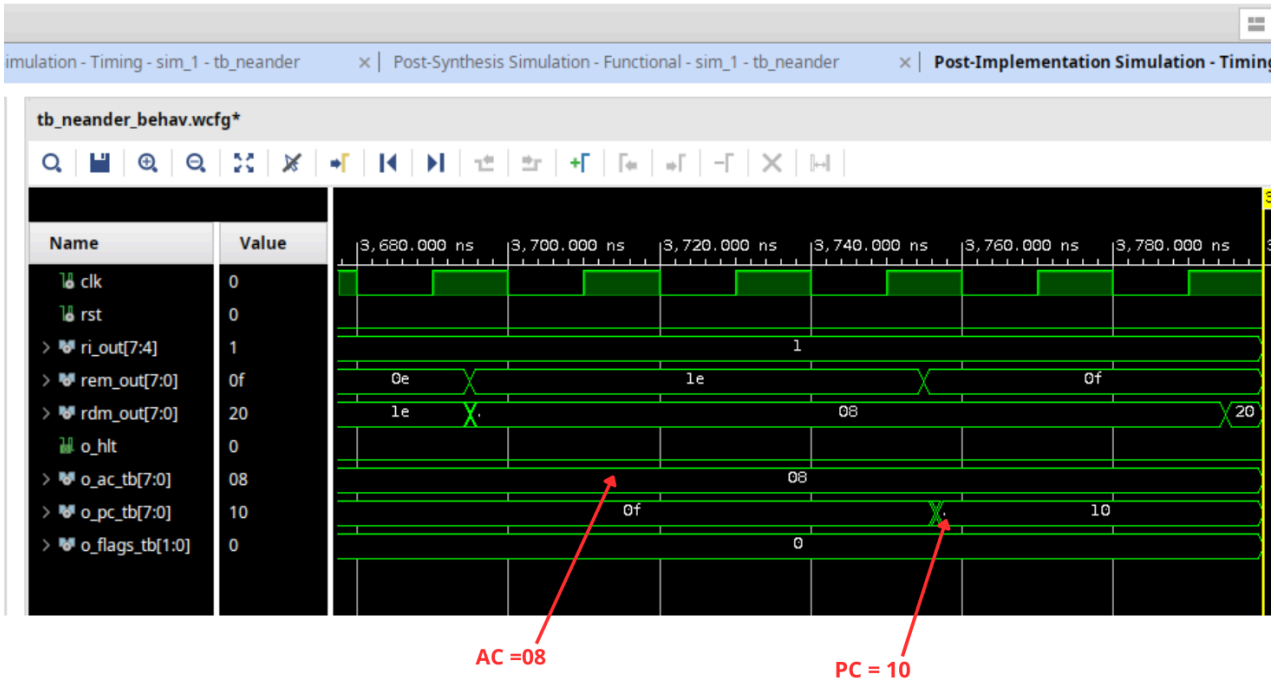
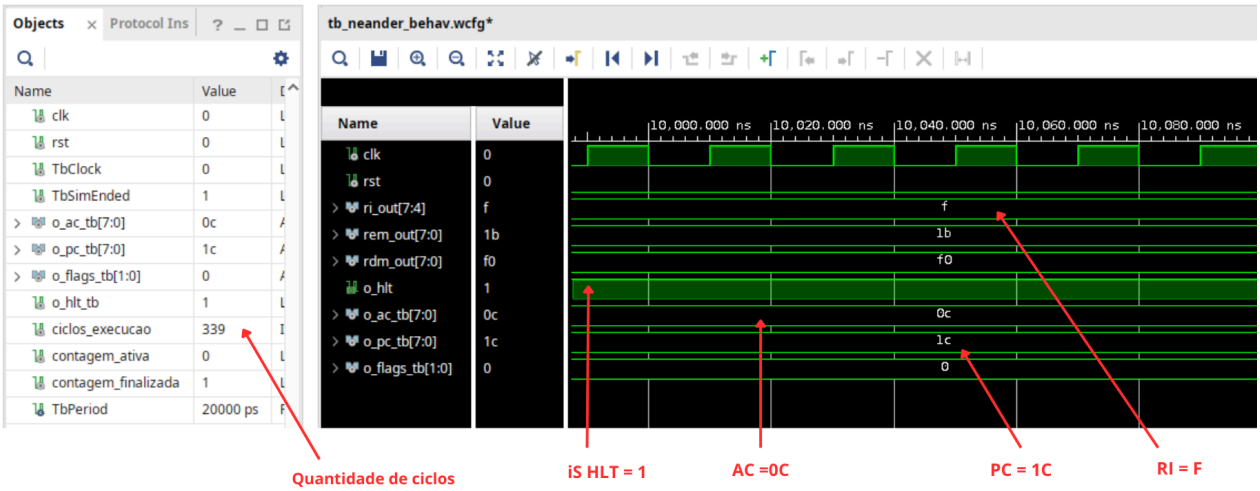


Figura 12 — Finalização da simulação temporal pós-implementação da multiplicação, com resultado 0C no acumulador e reconhecimento da instrução HLT.




6) Dados de área, tempo de execução em ciclos de relógio e tempo em segundos deve ser apresentado dado um determinado clock usado.

| Programa | Número de Instruções Executadas | Tempo de execução em # de ciclos de relógio (c.c.) | Tempo de execução em Segundos (Neander operando a 50 MHz) |
|---|---|---|---|
| Subtração de matrizes | 29 | 344 ciclos | 0,00000686 s |
| Multiplicação por somas sucessivas | 30 | 339 ciclos | 0,00000678 s |

**Dados de Área do Neander**

FPGA device: xc7s100fgga676-2
Numero de 4-LUTs: 22
Slice LUTs utilizadas: 95
Numero de ffps: 44
Numero de BRAM: 1 (RAMB18)
Numero de DSP 0

**Conclusão final:**

As simulações funcionais permitiram verificar a execução dos dois programas desenvolvidos. Na aplicação de subtração de matrizes, o processador realizou as operações elemento a elemento e obteve a matriz resultado esperada: 00 04 02, 02 FF FD e 02 01 02, em hexadecimal. Na aplicação de multiplicação por somas sucessivas, utilizando A = 04 e B = 03, o resultado obtido foi 0C, equivalente a 12 em decimal.

Também foram realizadas simulações temporais pós-implementação no Vivado para os dois programas, confirmando os resultados esperados, sendo essas as simulações com atraso que foram possíveis de realizar por meio da plataforma.

A síntese e a implementação física foram realizadas para o dispositivo FPGA `xc7s100fgga676-2`, com os recursos de hardware e tempos de execução apresentados nas tabelas deste relatório. A prototipação em placa física não foi realizada.

Anexos:
**Código completo tambem pode ser visto pelo GITHUB:**
**https://github.com/t-marti/Neander-VHDL-.git**

## Código ac.vhd

```
---8 bits
-- armazena o resultado das operações
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.NUMERIC_STD.ALL;

entity ac is
   PORT (
      clk,rst        : in  std_logic;
      data_mem       : in std_logic_vector(7 downto 0);
      data_ula       : in std_logic_vector(7 downto 0);
      op             : in std_logic_vector(1 downto 0);
      out_ac         : out std_logic_vector(7 downto 0)
   );
end entity ac;

architecture Behavioral of ac is
   signal data_ac  : std_logic_vector(7 downto 0);
begin
   process(clk, rst) begin
      if (rst = '1') then
         data_ac <= "00000000";
      elsif rising_edge(clk) then
         case op is
            when "00" =>
            data_ac <= data_mem;
            when "01" =>
            data_ac <= data_ula;
            when others =>

         end case;

      end if;
   end process;
   out_ac <= data_ac;
end Behavioral;
```

## Código controle.vhd

```
-- Unidade de controle: coordena a execucao das instrucoes do Neander.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity controle is
```

```
    port (
        -- Entradas gerais
        clk, rst                    : in  std_logic;

        -- Flags
        N_flag, Z_flag              : in  std_logic;

        -- Instrucoes identificadas pelo decoder
        is_nop, is_sta, is_lda      : in  std_logic;
        is_add, is_or, is_and       : in  std_logic;
        is_not, is_sub, is_jmp      : in  std_logic;
        is_jn, is_jz, is_hlt        : in  std_logic;

        -- Controle do PC
        select_PC                   : out std_logic;
        PC_mais_um                  : out std_logic;

        -- Controle do RI
        load_ri                     : out std_logic;

        -- Controle do AC
        op_ac                       : out std_logic_vector(1 downto 0);

        -- Controle da ULA
        op_ula                      : out std_logic_vector(2 downto 0);

        -- Controle das flags
        loadN                       : out std_logic;
        loadZ                       : out std_logic;

        -- Controle da BRAM
        mem_en                      : out std_logic;
        mem_we                      : out std_logic;

        -- Controle do REM e do mux do REM
        load_rem                    : out std_logic;
        sel_rem_mux                 : out std_logic;

        -- Controle do RDM e do mux do RDM
        load_rdm                    : out std_logic;
        sel_rdm_mux                 : out std_logic
    );
end entity controle;

architecture Behavioral of controle is

    -- Os nomes dos estados foram mantidos em portugues.
    -- Os estados adicionais separam os ciclos da BRAM sincrona.
    type state_type is (
```

```
        busca,
        leitura,
        carrega_rdm,
        carrega_ri,
        decodifica,
        busca_operando,
        leitura_operando,
        carrega_operando,
        busca_dado,
        leitura_dado,
        carrega_dado,
        prepara_escrita,
        escrita,
        execucao,
        retorno,
        parada
    );

    signal state, next_state : state_type;

begin

    ---------------------------------------------------------------------
    -- Registrador de estado
    ---------------------------------------------------------------------
    process(clk, rst)
    begin
        if rst = '1' then
            state <= busca;
        elsif rising_edge(clk) then
            state <= next_state;
        end if;
    end process;

    ---------------------------------------------------------------------
    -- Logica de proximo estado
    ---------------------------------------------------------------------
    process(state, is_nop, is_sta, is_lda, is_add, is_or,
            is_and, is_not, is_sub, is_jmp, is_jn, is_jz, is_hlt,
            N_flag, Z_flag)
    begin
        next_state <= state;

        case state is

            -- Busca do opcode
            when busca =>
                next_state <= leitura;
```

```
when leitura =>
   next_state <= carrega_rdm;

when carrega_rdm =>
   next_state <= carrega_ri;

when carrega_ri =>
   next_state <= decodifica;

-- Decodificacao da instrucao
when decodifica =>
   if is_hlt = '1' then
      next_state <= parada;

   elsif is_nop = '1' then
      next_state <= retorno;

   elsif is_not = '1' then
      next_state <= execucao;

   elsif is_sta = '1' or is_lda = '1' or
        is_add = '1' or is_or = '1' or
        is_and = '1' or is_sub = '1' or
        is_jmp = '1' or is_jn = '1' or is_jz = '1' then
      next_state <= busca_operando;

   else
      -- Opcode nao implementado: segue para a proxima instrucao.
      next_state <= retorno;
   end if;

-- Busca do segundo byte (endereco) da instrucao
when busca_operando =>
   next_state <= leitura_operando;

when leitura_operando =>
   next_state <= carrega_operando;

when carrega_operando =>
   if is_sta = '1' then
      next_state <= prepara_escrita;

   elsif is_lda = '1' or is_add = '1' or
        is_or = '1' or is_and = '1' or is_sub = '1' then
      next_state <= busca_dado;

   elsif is_jmp = '1' or
        (is_jn = '1' and N_flag = '1') or
        (is_jz = '1' and Z_flag = '1') then
```

```
            next_state <= execucao;

         else
            -- JN/JZ nao tomado: o PC ja foi incrementado
            -- durante a busca do byte de endereco.
            next_state <= retorno;
         end if;

      -- Acesso ao dado apontado pelo endereco lido
      when busca_dado =>
         next_state <= leitura_dado;

      when leitura_dado =>
         next_state <= carrega_dado;

      when carrega_dado =>
         next_state <= execucao;

      -- Escrita na memoria (STA)
      when prepara_escrita =>
         next_state <= escrita;

      when escrita =>
         next_state <= retorno;

      -- Execucao da operacao na ULA ou desvio
      when execucao =>
         next_state <= retorno;

      when retorno =>
         next_state <= busca;

      when parada =>
         next_state <= parada;

   end case;
end process;

-------------------------------------------------------------------
-- Geracao dos sinais de controle
-------------------------------------------------------------------
process(state, is_lda, is_add, is_sub, is_and, is_or,
      is_not, is_jmp, is_jn, is_jz, N_flag, Z_flag)
begin
   -- Valores padrao: nenhum registrador carrega e a memoria fica inativa.
   select_PC    <= '0';
   PC_mais_um   <= '0';
   load_ri      <= '0';
```

```
-- op_ac = 00: AC recebe data_mem (RDM)
-- op_ac = 01: AC recebe data_ula
-- op_ac = 10/11: AC mantem o valor
op_ac       <= "10";

-- op_ula = 000 ADD; 001 SUB; 010 AND; 011 OR;
--          100 NOT; 101 passagem de RDM (LDA)
op_ula      <= "000";

loadN       <= '0';
loadZ       <= '0';

mem_en      <= '0';
mem_we      <= '0';

load_rem    <= '0';
sel_rem_mux <= '0'; -- 0: PC; 1: RDM

load_rdm    <= '0';
sel_rdm_mux <= '0'; -- 0: saida da BRAM; 1: AC

case state is

   -- REM <- PC; PC <- PC + 1
   when busca =>
      load_rem    <= '1';
      sel_rem_mux <= '0';
      PC_mais_um  <= '1';

   -- A BRAM recebe o endereco que ja esta armazenado no REM.
   when leitura =>
      mem_en <= '1';

   -- RDM <- saida da BRAM
   when carrega_rdm =>
      load_rdm <= '1';

   -- RI <- RDM
   when carrega_ri =>
      load_ri <= '1';

   -- REM <- PC; PC <- PC + 1 para buscar o segundo byte
   when busca_operando =>
      load_rem    <= '1';
      sel_rem_mux <= '0';
      PC_mais_um  <= '1';

   when leitura_operando =>
      mem_en <= '1';
```

```
-- RDM <- segundo byte (endereco da instrucao)
when carrega_operando =>
   load_rdm <= '1';

-- REM <- RDM (endereco do dado)
when busca_dado =>
   load_rem    <= '1';
   sel_rem_mux <= '1';

when leitura_dado =>
   mem_en <= '1';

-- RDM <- MEM(REM)
when carrega_dado =>
   load_rdm <= '1';

-- REM <- RDM e RDM <- AC para preparar STA
when prepara_escrita =>
   load_rem    <= '1';
   sel_rem_mux <= '1';

   load_rdm    <= '1';
   sel_rdm_mux <= '1';

-- MEM(REM) <- RDM
when escrita =>
   mem_en <= '1';
   mem_we <= '1';

-- Execucao da instrucao
when execucao =>
   if is_lda = '1' then
      op_ula <= "101"; -- ULA encaminha RDM para a saida
      op_ac  <= "01";
      loadN  <= '1';
      loadZ  <= '1';

   elsif is_add = '1' then
      op_ula <= "000";
      op_ac  <= "01";
      loadN  <= '1';
      loadZ  <= '1';

   elsif is_sub = '1' then
      op_ula <= "001";
      op_ac  <= "01";
      loadN  <= '1';
      loadZ  <= '1';
```

```
            elsif is_and = '1' then
               op_ula <= "010";
               op_ac  <= "01";
               loadN  <= '1';
               loadZ  <= '1';

            elsif is_or = '1' then
               op_ula <= "011";
               op_ac  <= "01";
               loadN  <= '1';
               loadZ  <= '1';

            elsif is_not = '1' then
               op_ula <= "100";
               op_ac  <= "01";
               loadN  <= '1';
               loadZ  <= '1';

            elsif is_jmp = '1' or
                 (is_jn = '1' and N_flag = '1') or
                 (is_jz = '1' and Z_flag = '1') then
               select_PC <= '1'; -- PC <- RDM
            end if;

         when others =>
            null;

      end case;
   end process;

end architecture Behavioral;
```

**Código decoder.vhd**

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.NUMERIC_STD.ALL;

entity decoder is
   PORT (
      opcode       : in std_logic_vector(3 downto 0);
      is_nop       : out std_logic; -- 0000 não faz nada
      is_sta       : out std_logic; -- 0001 MEM(end) <- AC
      is_lda       : out std_logic; -- 0010 AC <- MEM(end)
      is_add       : out std_logic; -- 0011 AC <- MEM(end) + AC
      is_or        : out std_logic; -- 0100 AC <- MEM(end) OR AC
      is_and       : out std_logic; -- 0101 AC <- MEM(end) AND AC
      is_not       : out std_logic; -- 0110 AC <- NOT AC
```

```
        is_sub        : out std_logic;  -- 0111
        is_jmp        : out std_logic; -- 1000 PC <- end
        is_jn         : out std_logic; -- 1001 Se N=1, PC <- end
        is_jz         : out std_logic; -- 1010 Se Z=1, PC <- end
        is_hlt        : out std_logic -- 1111 Para o processamento

    );
end entity decoder;

architecture Behavioral of decoder is

begin
    process(opcode) begin
        -- Todas as saídas começam em zero
        is_nop <= '0';
        is_sta <= '0';
        is_lda <= '0';
        is_add <= '0';
        is_or  <= '0';
        is_and <= '0';
        is_not <= '0';
        is_sub <= '0';
        is_jmp <= '0';
        is_jn  <= '0';
        is_jz  <= '0';
        is_hlt <= '0';

        case opcode is
            when "0000" =>
                is_nop <= '1';
            when "0001" =>
                is_sta <= '1';
            when "0010" =>
                is_lda <= '1';
            when "0011" =>
                is_add <= '1';
            when "0100" =>
                is_or <= '1';
            when "0101" =>
                is_and <= '1';
            when "0110" =>
                is_not <= '1';
            when "0111" =>
                is_sub <= '1';
            when "1000" =>
                is_jmp <= '1';
            when "1001" =>
                is_jn <= '1';
            when "1010" =>
```

```
            is_jz <= '1';
         when "1111" =>
            is_hlt <= '1';
         when others =>
            null;
      end case;
   end process;

end architecture Behavioral;
```

**Código Flags**

```
-- armazena os indicadores negativo e zero

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

entity flags is
   PORT (
      clk,rst          : in  std_logic;
      N_in, Z_in       : in std_logic;
      loadN, loadZ     : in std_logic;
      Z_out, N_out     : out std_logic
   );
end entity flags;


architecture Behavioral of flags is
signal data_N, data_Z: std_logic;
begin
   process(clk, rst) begin
      if (rst = '1') then
         data_N <= '0';
         data_Z <= '0';
      elsif rising_edge(clk) then
         if(loadZ = '1') then
            data_Z <= Z_in;
         end if;

         if(loadN ='1') then
            data_N <= N_in;
         end if;
      end if;
   end process;
   N_out <= data_N;
   Z_out <= data_Z;
end Behavioral;
```

## Código MUX

```
--mux 2:1 com 8 bits
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.NUMERIC_STD.ALL; -- para as op aritméticas (+ e -)

entity mux2p1_8bits is
   PORT (
      a            : IN  std_logic_vector(7 DOWNTO 0);
      b            : IN  std_logic_vector(7 DOWNTO 0);
      sel_mux      : IN  std_logic;                    -- Seletor de operação
      mux_out      : OUT std_logic_vector(7 downto 0)
   );
end entity mux2p1_8bits;

architecture Behavioral of mux2p1_8bits is

begin
   with sel_mux select
      mux_out <= a when '0',
                 b when '1',
                 (others => '0') when others;

end architecture Behavioral;
```

Código PC.vhd

```
--8 bits
--o PC recebe um sinal e se o sinal foi x = avançar a posição atual +1
--se o sinal for y = avançar para uma posição z
-- ou não faz nada
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.NUMERIC_STD.ALL; -- para as op aritméticas (+ e -)

entity PC is
   PORT (
      clk,rst      : in  std_logic;
      select_PC    : in std_logic;
      PC_mais_um   : in std_logic;
      in_pc        : in std_logic_vector(7 downto 0);
      out_pc       : out std_logic_vector(7 downto 0)
   );
end entity PC;

architecture Behavioral of PC is
   signal dataPC  : std_logic_vector(7 downto 0);
begin
```

```
   process(clk, rst) begin
      if (rst = '1') then
         dataPC <= "00000000";
      elsif rising_edge(clk) then
         if (select_PC ='1') then
            dataPC <= in_pc;
         elsif (PC_mais_um = '1') then
            dataPC <= std_logic_vector(unsigned(dataPC)+1);
         end if;
      end if;
   end process;
   out_pc <= dataPC;
end Behavioral;
```

**Código RDM.vhd**

```
--guarda os dados lidos ou escritos
-- guarda o end acessado na memoria
-- registrador de endereço da memoria
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;


entity reg_rdm is
   PORT (
      clk, rst          : in  std_logic;
      data_in           : in std_logic_vector(7 downto 0);
      load_rdm          : in std_logic;
      out_rdm           : out std_logic_vector(7 downto 0)
   );
end entity reg_rdm;

architecture Behavioral of reg_rdm is
   signal data_rdm  : std_logic_vector(7 downto 0);
begin
   process(clk, rst) begin
      if (rst = '1') then
         data_rdm <= "00000000";
      elsif rising_edge(clk) then
            if(load_rdm= '1') then
               data_rdm <= data_in;
            end if;
      end if;
   end process;
   out_rdm <= data_rdm;
end Behavioral;
```

## Código REM.vhd

```
-- guarda o end acessado na memoria
-- registrador de endereço da memoria
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;


entity reg_rem is
   PORT (
      clk, rst         : in  std_logic;
      data_in          : in std_logic_vector(7 downto 0);
      load_rem         : in std_logic;
      out_rem          : out std_logic_vector(7 downto 0)
   );
end entity reg_rem;

architecture Behavioral of reg_rem is
   signal data_rem  : std_logic_vector(7 downto 0);
begin
   process(clk, rst) begin
      if (rst = '1') then
         data_rem <= "00000000";
      elsif rising_edge(clk) then
            if(load_rem= '1') then
               data_rem <= data_in;
            end if;
      end if;
   end process;
   out_rem <= data_rem;
end Behavioral;
```

## Código ri.vhd

```
-- guarda o end acessado na memoria
-- registrador de endereço da memoria
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;


entity reg_rem is
   PORT (
      clk, rst         : in  std_logic;
      data_in          : in std_logic_vector(7 downto 0);
      load_rem         : in std_logic;
      out_rem          : out std_logic_vector(7 downto 0)
   );
end entity reg_rem;
```

```
architecture Behavioral of reg_rem is
   signal data_rem  : std_logic_vector(7 downto 0);
begin
   process(clk, rst) begin
      if (rst = '1') then
         data_rem <= "00000000";
      elsif rising_edge(clk) then
            if(load_rem= '1') then
               data_rem <= data_in;
            end if;
      end if;
   end process;
   out_rem <= data_rem;
end Behavioral;
```